

**MEDIATEK**

# MT8668 Yocto Audio User Manual

**版本:** 1.0  
**出版日期:** 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

---

版本记录

| 版本  | 日期         | 作者 | 描述  |
|-----|------------|----|-----|
| 1.0 | 2026-01-28 | 程诚 | 正式版 |

目录

版本记录 ..... 2

目录 ..... 3

图片目录 ..... 4

表格目录 ..... 4

1 音频 ..... 5

    1.1 概述 ..... 5

        1.1.1 简单介绍 ..... 5

        1.1.2 名词解释 ..... 5

    1.2 架构/流程概述 ..... 5

        1.2.1 硬件 ..... 5

        1.2.2 软件 ..... 9

        1.2.3 音频驱动 ..... 10

    1.3 配置/客制指南 ..... 14

        1.3.1 内核设备树 ..... 14

        1.3.2 音频设备配置 ..... 16

    1.4 Yocto 播放 ADSP 音频 ..... 16

        1.4.1 软件架构 ..... 16

        1.4.2 测试命令 ..... 17

    1.5 常见问题/故障排除 ..... 18

        1.5.1 如何测试音频驱动? ..... 18

        1.5.2 如何开启 log? ..... 19

        1.5.3 Audio Register 的查看 ..... 19

        1.5.4 dts/kconfig/ko table/dws 文件的查看 ..... 19

附件一 附加条款 ..... 21

图片目录

图 1-1. 音频系统.....6

图 1-2. AFE 框图 .....8

图 1-3. Yocto 音频系统.....9

图 1-4. 音频驱动架构.....10

图 1-5. 音频驱动组件.....11

图 1-6. 音频设备.....12

图 1-7. Audio CPU DAI.....12

图 1-8. DPCM 设备 .....13

图 1-9. MTK Audio DTS.....15

图 1-10. 播放音乐的音频路径.....16

表格目录

表 1-1. 名词解释.....5

# 1 音频

## 1.1 概述

### 1.1.1 简单介绍

本章节介绍 MT8668 Audio 的硬件、软件和基本功能。

### 1.1.2 名词解释

表 1-1. 名词解释

| 缩写  | 解释                                                                             |
|-----|--------------------------------------------------------------------------------|
| I2S | Integrated Interchip Sound。是飞利浦在 1986 年定义（1996 年修订）的数字音频传输标准，用于数字音频数据在器件之间传输。  |
| PCM | Pulse Code Modulation。通过等时间隔（即采样率时钟周期）采样将模拟信号数字化的方法。PCM 数字音频接口，传输的即是 PCM 格式数据。 |
| TDM | Time Division Multiplexin。TDM 数字音频接口，可以传输多声道数据。                                |
| AFE | Audio Front End                                                                |
| DAI | Digital Audio Interface                                                        |
| DTS | Device Tree Source                                                             |

## 1.2 架构/流程概述

### 1.2.1 硬件

MT8668 的 Audio System 的硬件整体组成如图 1-1 所示：

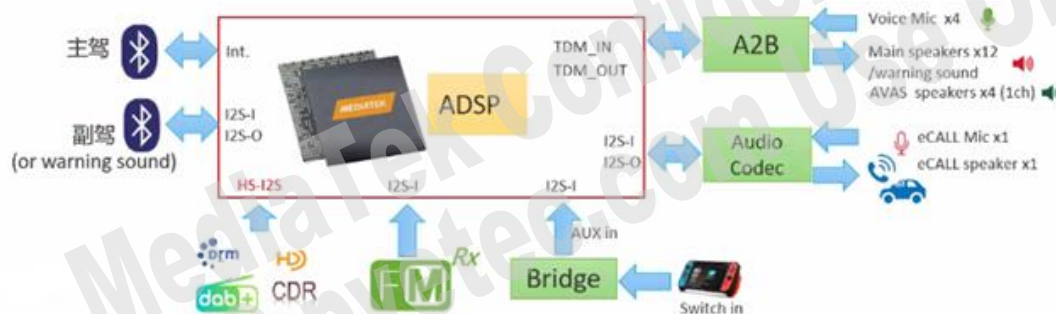
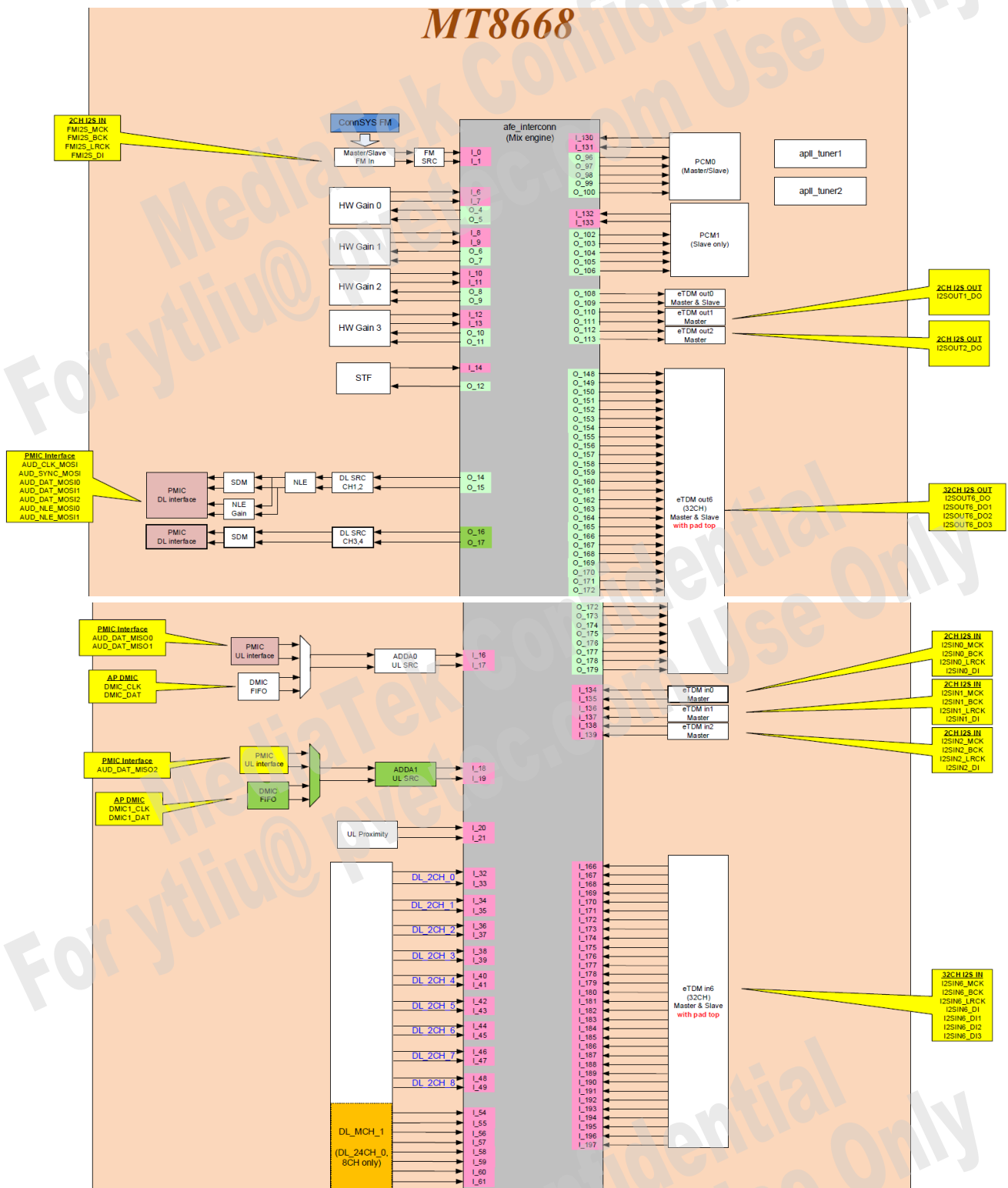


图 1-1. 音频系统

Audio Hardware 包括对外的硬件接口、AFE\_Interconn、AFE\_Tinyconn 和 AFE\_Memif，以及后端的 Codec。其中，对外的硬件接口如下：

- 1 x TDM32(O) + 1 x TDM32(I) (48K/32b)
  - 1 x TDM 32ch OUT, master/slave, for Main SPK/warning sound/AVAS SPK, TDM OUT: 1/2/4 lanes (share pin)
  - 1 x TDM 32ch IN, master/slave, for microphones and echo reference, TDM32 IN port can share or split clock with corresponding TDM32 OUT port, TDM IN: 1/2/4 lanes (share pin)
  - GASRC up to 16ch (TDM slave up to 16ch)
- 2 x I2S(O) 2ch + 4 x I2S(I) 2ch (192K/32b)
  - 1 x I2S Bi-directional, master, for eCall in/out
  - 1 x I2S Bi-directional, master, for BT voice/ warning sound
  - 1 x I2S IN, master, for AUX IN
  - 1 x I2S IN, master/slave, for AM/FM radio

其接口细节参考图 1-2。





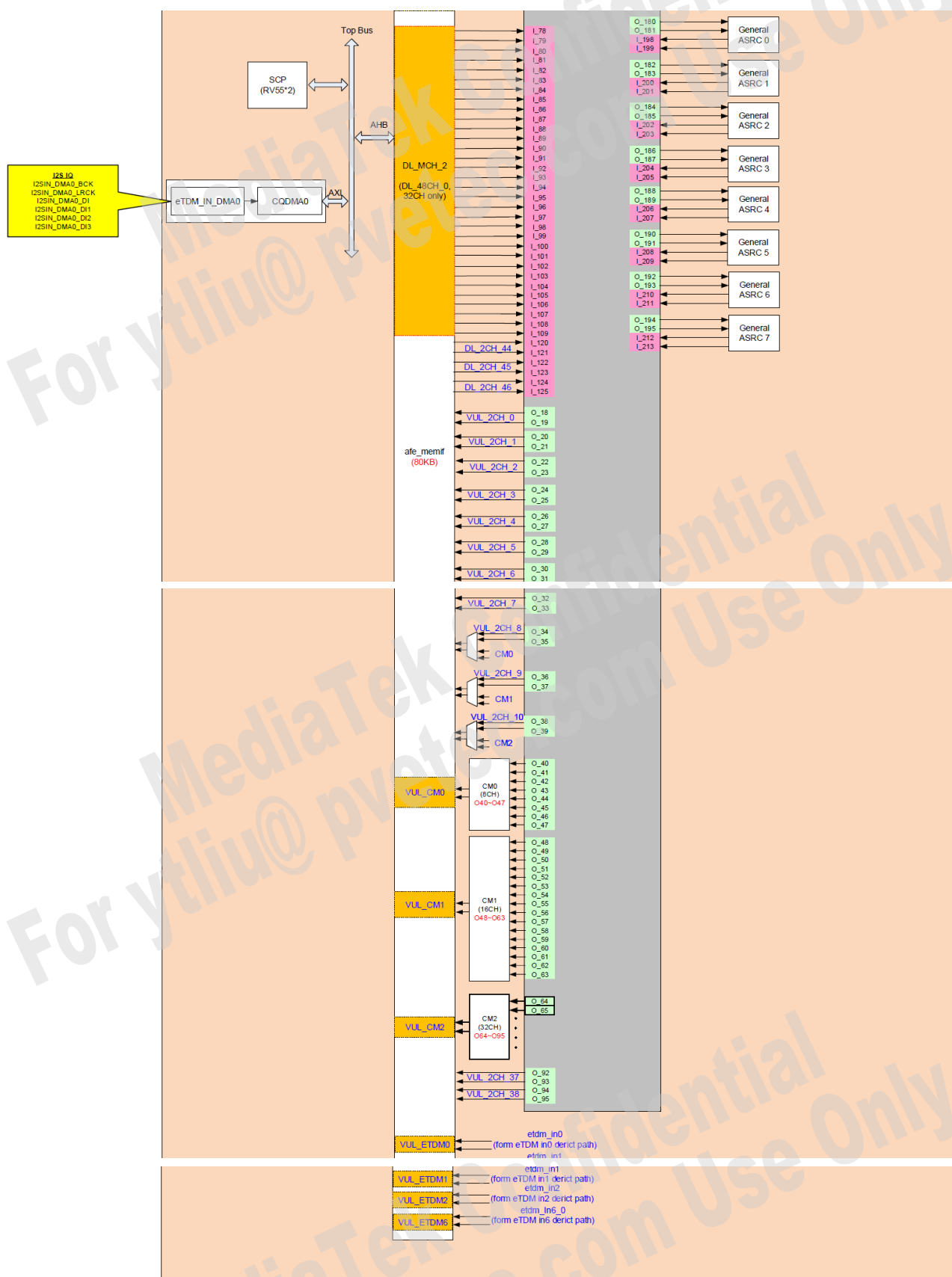


图 1-2. AFE 框图



而内部的 AFE 硬件接口可以归纳为两类：

- Memory interfaces
  - 即 AFE\_memif。这部分相当于 DMA，主要负责将上层应用 Playback 的 PCM 数据搬运到指定的 DL (downlink) port 输出，或者将 Capture 的 PCM 数据从指定的 UL (uplink) port 捕获后传输给上层应用。
- Connect interfaces
  - 有两种，分别是 AFE\_interconn 和 AFE\_tiny\_conn。两者的主要区别是支持的数据 bit width 不同，前者最大只支持 24bit data，而后者可以支持到 32bit。两者的作用是相同的，主要是负责将 Memif 的 DL 或者 UL 和后端的 I2S/TDM 等硬件接口连接起来，构建一条和外部设备传输数据的 I/O 通路。对 Playback 而言，即是 将 DL 的数据送到指定的 Output port (即 I2S/TDM out 等) 输出；反之，对 Capture 而言，将 UL 接口和指定的 Input port (即 I2S/TDM in 等) 连通，捕获外部设备输入的数据。另外，中间的 I/O 通路还可以添加 HW SRC、Gain control 以及 mix 等功能。

## 1.2.2 软件

MT8668 基于 ALSA 接口调用的架构图 1-3 所示：

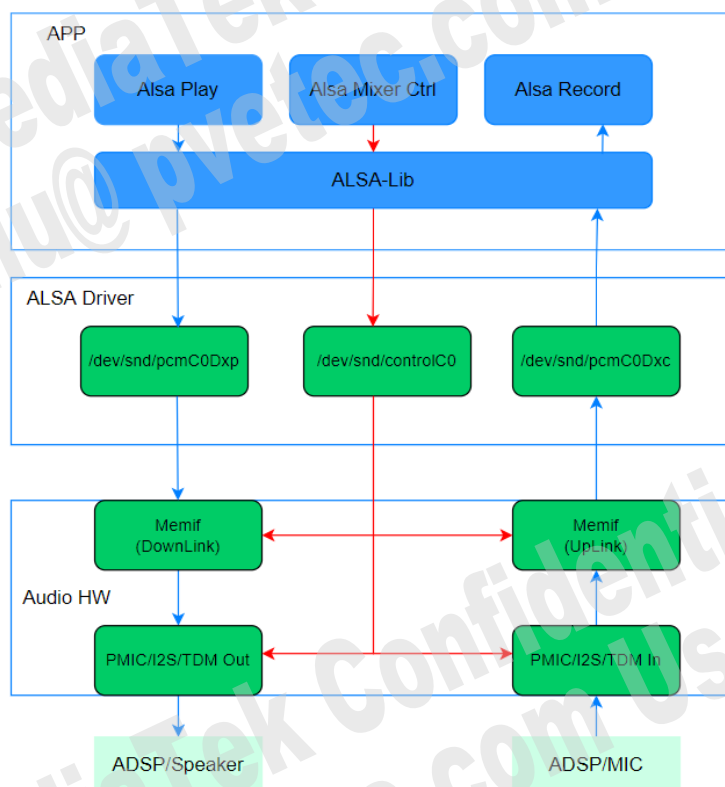


图 1-3. Yocto 音频系统

ALSA Lib 模块是 Linux 原生的音频模块。这部分相关的资料可以在网上查询了解，这里就不再赘述。本文档主要是讲解 MTK 专有的 Audio Driver。

### 1.2.3 音频驱动

Audio Driver 是标准的 ALSA ASoC Architecture:

- **Machine** 是指具体的某一款机器，或者开发板。由此可以看出 Machine 几乎是不可复用的，每个 Machine 上的硬件实现可能都不一样，CPU 不一样，Codec 不一样，音频的输入、输出设备也不一样，Machine 为 CPU、Codec、输入输出设备提供了一个载体。MT8668 公板就是一个 Machine，客户定制的开发板也是一个 Machine。
- **Platform** 一般是指某一个 SoC 平台，比如 MT8668。与音频相关的通常包含该 SoC 中的时钟、DMA、I2S、PCM 等等，只要指定了 SoC，那么我们可以认为它会有一个对应的 Platform，它只与 SoC 相关，与 Machine 无关。上面所讲的 AFE 就属于 Platform。
- **Codec** 一般指 I2S/TDM 接口、D/A、A/D、Mixer、PA（功放）等，通常包含多种输入（Mic、Line-in、I2S、PCM）和多个输出（耳机、喇叭、听筒，Line-out）。Codec 也是可复用的部件，同一个 Codec 可以被不同的 Machine 使用。MT8668 公板对应的 Codec 有 AK7709、ES8311，客户使用的外部 DSP 也属于一种 Codec。

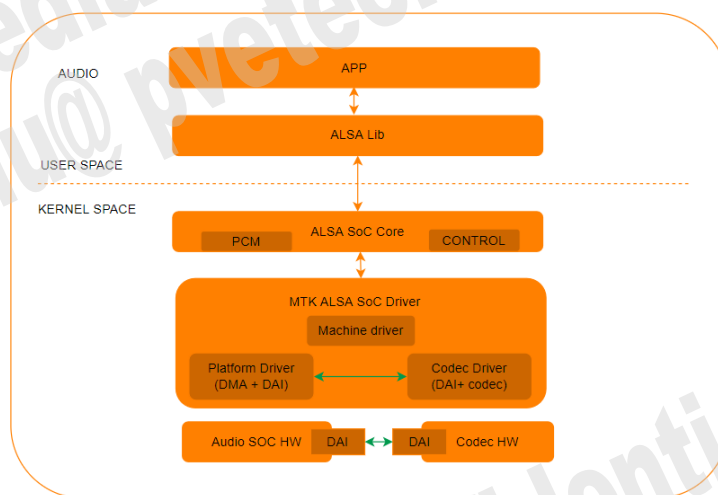


图 1-4. 音频驱动架构

对应于上述的三种 components，也有相应的驱动：

- **Machine Driver** 负责处理机器特有的一些控件和音频事件（例如，当播放音频时，需要先行打开一个放大器）；单独的 Platform 和 Codec 驱动是不能工作的，它必须由 Machine 驱动把它们结合在一起才能完成整个设备的音频处理工作。

- **Platform Driver** 它包含了该 SoC 平台的音频 DMA 和音频接口的配置和控制（I2S，PCM，等）；它也不能包含任何与板子或机器相关的代码。
- **Codec Driver** ASoC 中的一个重要设计原则就是要求 Codec 驱动是平台无关的，它包含了一些音频的控件（Controls），音频接口，DAMP（动态音频电源管理）的定义和某些 Codec IO 功能。为了保证硬件无关性，任何特定于平台和机器的代码都要移到 Platform 和 Machine 驱动中。

MT8668 平台对应的 Audio Driver 模块如下：

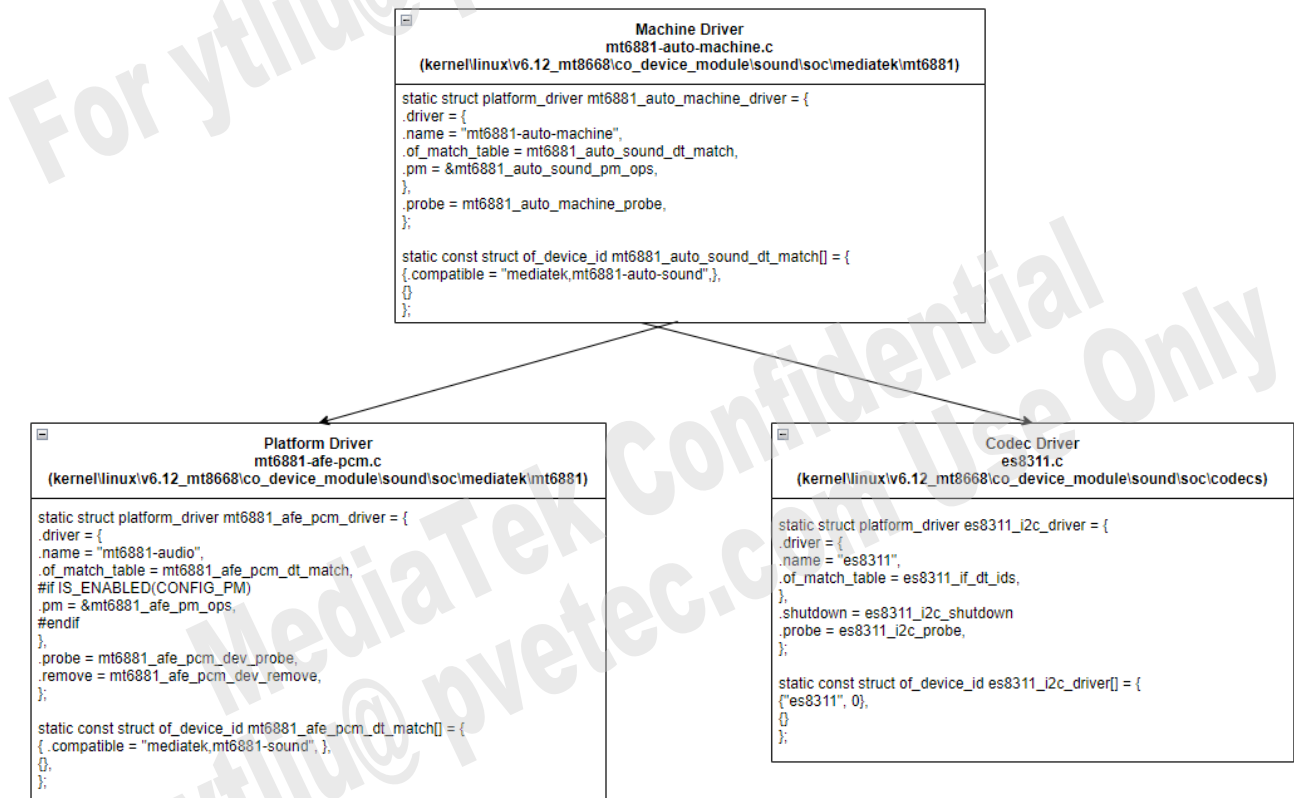


图 1-5. 音频驱动组件

Audio Driver 的这些组件在系统启动后，Kernel 初始化的过程中相继完成初始化并生成相应的驱动设备。系统启动完成后，可以在 adb shell 的 console 界面查看相应的设备信息：

```

root@auto8668p164sos:/# cat /proc/asound/cards
0 [mt6881sound ]: mt6881-sound - mt6881-sound
                                mt6881-sound
root@auto8668p164sos:/# ls /sys/kernel/debug/asoc/mt6881-sound
11050000.mt6881-afe-pcm Hostless_BT I2SOUT2
18830000.mtk-btcvsd-snd Hostless_FM I2SOUT6
Capture_1 Hostless_HW_Gain_Audio Playback_1
Capture_10 Hostless_HW_SRC_1_IN Playback_11
Capture_11 Hostless_HW_SRC_1_OUT Playback_12
Capture_12 Hostless_HW_SRC_2_IN Playback_2
Capture_14 Hostless_HW_SRC_2_OUT Playback_3
Capture_2 Hostless_HW_SRC_3_IN Playback_44
Capture_3 Hostless_HW_SRC_3_OUT Playback_45
Capture_38 Hostless_LPBK Playback_46
Capture_4 Hostless_SRC_1 Playback_48
Capture_5 Hostless_SRC_3 Playback_5
Capture_6 Hostless_SRC_AAudio Playback_6
Capture_7 Hostless_SRC_Bargein Playback_7
Capture_8 Hostless_Speech Playback_8
Capture_9 Hostless_Sph_Echo_Ref Playback_9
Capture_Mono_1 Hostless_Spk_Init dapm
Capture_Mono_2 I2SIN2 dapm_pop_time
Hostless_ADDA_DL_I2S_OUT I2SIN6 snd-audio-dsp
root@auto8668p164sos:/# ls /sys/kernel/debug/asoc/
components dais mt6881-sound
root@auto8668p164sos:/# cat /sys/kernel/debug/asoc/components
18830000.mtk-btcvsd-snd
11050000.mt6881-afe-pcm
snd-audio-dsp
snd-soc-dummy
snd-soc-dummy
root@auto8668p164sos:/#

```

图 1-6. 音频设备

按照 DPCM 架构，ALSA Driver 将 Memory Interfaces 虚拟成 Front End (FE) 设备，将对外的 Hardware Interfaces 虚拟成 Back End (BE) 设备。FE 和 BE 通过 Connection Interfaces 串接起来，形成一条 Audio path，即可用来进行 Audio 数据的传输。因为 FE 和 BE 都是 MT8668 SoC 里的 Audio Interfaces，所以，ALSA Driver 将它们统一作为 CPU DAI 来看待。而更后端的 PMIC 或者外部 DSP 就是 Codec DAI。

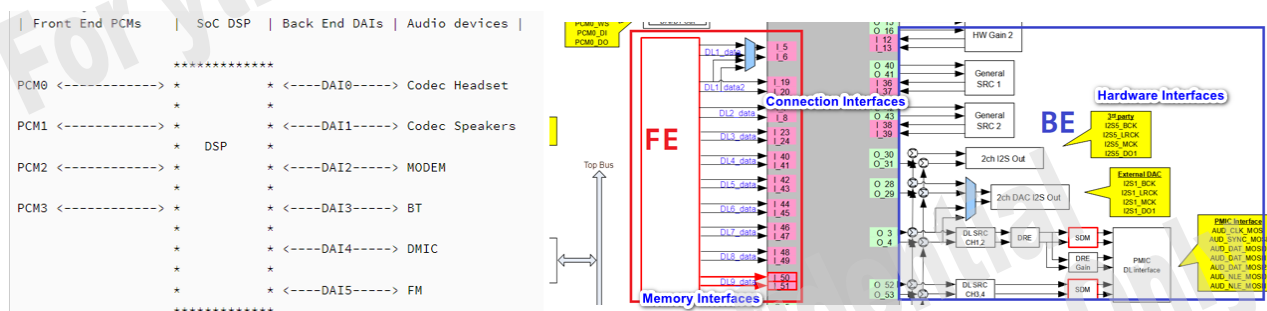


图 1-7. Audio CPU DAI

ALSA Driver 会构建一系列的 Control/Mixer APIs 来控制 DAI 并打通 Audio Path。而用来控制这些 APIs 的设备即是 Control 设备。同样，用来控制具体的 FE 并读写数据的设备节点即是 PCM 设备。

图 1-8 是 MT8668 平台的 Control 设备节点和 PCM 设备节点：

```

root@auto8668p164sos:/# ls /dev/snd/
by-path      pcmC0D11p    pcmC0D26c    pcmC0D35p    pcmC0D43p    pcmC0D78p    pcmC0D92p
controlC0    pcmC0D12p    pcmC0D27c    pcmC0D36p    pcmC0D44c    pcmC0D79p    pcmC0D93p
pcmC0D0p     pcmC0D13p    pcmC0D28c    pcmC0D37c    pcmC0D44p    pcmC0D7p     pcmC0D94p
pcmC0D100p   pcmC0D14c    pcmC0D29c    pcmC0D37p    pcmC0D45c    pcmC0D80p    pcmC0D95p
pcmC0D101p   pcmC0D15c    pcmC0D2p     pcmC0D38c    pcmC0D45p    pcmC0D81p    pcmC0D96p
pcmC0D102p   pcmC0D16c    pcmC0D30c    pcmC0D38p    pcmC0D4p     pcmC0D82c    pcmC0D97p
pcmC0D103p   pcmC0D17c    pcmC0D30p    pcmC0D39c    pcmC0D5p     pcmC0D83p    pcmC0D98p
pcmC0D104p   pcmC0D18c    pcmC0D31c    pcmC0D39p    pcmC0D6p     pcmC0D84c    pcmC0D99p
pcmC0D105p   pcmC0D19c    pcmC0D31p    pcmC0D3p     pcmC0D72c    pcmC0D85p    pcmC0D9p
pcmC0D106p   pcmC0D1p     pcmC0D32c    pcmC0D40c    pcmC0D73c    pcmC0D86c    timer
pcmC0D107p   pcmC0D20c    pcmC0D32p    pcmC0D40p    pcmC0D73p    pcmC0D87p
pcmC0D108p   pcmC0D21c    pcmC0D33c    pcmC0D41c    pcmC0D74c    pcmC0D88c
pcmC0D109p   pcmC0D22c    pcmC0D33p    pcmC0D41p    pcmC0D74p    pcmC0D89p
pcmC0D10p    pcmC0D23c    pcmC0D34c    pcmC0D42c    pcmC0D75p    pcmC0D8p
pcmC0D110p   pcmC0D24c    pcmC0D34p    pcmC0D42p    pcmC0D76p    pcmC0D90p
pcmC0D111p   pcmC0D25c    pcmC0D35c    pcmC0D43c    pcmC0D77p    pcmC0D91p
root@auto8668p164sos:/# cat /proc/asound/pcm
00-00: DL0 (*) : : playback 1
00-01: DL1 (*) : : playback 1
00-02: DL2 (*) : : playback 1
00-03: DL3 (*) : : playback 1
00-04: DL4 (*) : : playback 1
00-05: DL5 (*) : : playback 1
00-06: DL6 (*) : : playback 1
00-07: DL7 (*) : : playback 1
00-08: DL8 (*) : : playback 1
00-09: DL44 (*) : : playback 1
00-10: DL45 (*) : : playback 1
00-11: DL46 (*) : : playback 1
00-12: DLM1 (*) : : playback 1
00-13: DLM2 (*) : : playback 1
00-14: UL0 (*) : : capture 1
00-15: UL1 (*) : : capture 1
00-16: UL2 (*) : : capture 1
00-17: UL3 (*) : : capture 1
00-18: UL4 (*) : : capture 1
00-19: UL5 (*) : : capture 1
00-20: UL6 (*) : : capture 1
00-21: UL7 (*) : : capture 1
00-22: UL8 (*) : : capture 1
00-23: UL9 (*) : : capture 1
00-24: UL10 (*) : : capture 1
00-25: UL37 (*) : : capture 1
00-26: UL38 (*) : : capture 1

```

图 1-8. DPCM 设备

同时上图也显示了 PCM 节点和 AFE 的对应关系。

在播放音乐等 playback 的场景下，应用层就是通过往 Playback 的 PCM 节点写入 Audio 数据，传送到后端的喇叭或者功放来放出声音；反之，在录音等 Capture 的场景下，应用层从 Capture 的 PCM 节点读取 Audio 数据，获得从外部 MIC 等设备传入的 Audio 数据。

## 1.3 配置/客制指南

### 1.3.1 内核设备树

ALSA Driver 的配置主要是在 DTS 里，里面包含 Audio 相关的 Machine/Platform/Codec 等 Driver 的设备节点配置，包括 Register address、memory address、Interrupt number 等。

MT8668 的 DTS 配置文件位于

`src/kernel/linux/v6.12_mt8668/co_device_module/arch/arm64/boot/dts/mediatek/`。主要的配置文件如下：

`mt8668.dts`

`auto8668p1_64.dts`

相应的 Machine/Platform/Codec 的配置如下：



```

&sound {
    compatible = "mediatek,mt6881-auto-sound";
    dai-link-0 {
        link-name = "I2SOUT6";
        mediatek,clk-provider = "cpu";
        codec {
            sound-dai = <ak7709>;
        };
    };

    dai-link-1 {
        link-name = "I2SIN6";
        mediatek,clk-provider = "cpu";
        codec {
            sound-dai = <ak7709>;
        };
    };

    dai-link-2 {
        link-name = "I2SOUT2";
        mediatek,clk-provider = "cpu";
        codec {
            sound-dai = <es8311>;
        };
    };
};

afe: mt6881-afe-pcm@11050000 {
    compatible = "mediatek,mt6881-sound";
    reg = <0 0x11050000 0 0x10000>;
    interrupts = <GIC_SPI 520 IRQ_TYPE_LEVEL_HIGH 0>;
    topckgen = <apmixedsys_reg_clk>;
    apmixedsys = <apmixedsys_clk>;
    power-domains = <scpsys MT6881_POWER_DOMAIN_AUDIO>;
    clocks = <afe_clk CLK_AFE_APLL1_AUDIO>,
        <afe_clk CLK_AFE_APLL2_AUDIO>,
        <afe_clk CLK_AFE_APLL_TUNER1_AUDIO>,
        <afe_clk CLK_AFE_APLL_TUNER2_AUDIO>,
        <afe_clk CLK_AFE_AUDIO_F26M_AUDIO>,
        <afe_clk CLK_AFE_AUDIO_HOPPING_AUDIO>,
        <afe_clk CLK_AFE_DL0_DAC_AUDIO>,
        <afe_clk CLK_AFE_DL0_DAC_HIRES_AUDIO>,
        <afe_clk CLK_AFE_DL0_PREDIS_AUDIO>,
        <afe_clk CLK_AFE_DL1_DAC_AUDIO>,
        <afe_clk CLK_AFE_DL1_DAC_HIRES_AUDIO>,
        <afe_clk CLK_AFE_DL1_PREDIS_AUDIO>,
        <afe_clk CLK_AFE_UL0_ADC_AUDIO>,
        <afe_clk CLK_AFE_UL0_ADC_HIRES_AUDIO>,
        <afe_clk CLK_AFE_UL1_ADC_AUDIO>,
        <afe_clk CLK_AFE_UL1_ADC_HIRES_AUDIO>,
        <afe_clk CLK_AFE_CQDMA_BUS_AUDIO>,

```

图 1-9. MTK Audio DTS

当 Kernel 初始化时，Audio Driver 的各个组件可以根据 DTS 里的配置，得到相应的 register address、memory address、clock configuration、GPIO setting 等参数。在 Audio Driver 使用的过程中，就可以使用这些参数完成 Audio



Hardware 的设置，打通 Audio Path。所以，客户平台在开发时，也要根据实际硬件的情况定制相对应的 DTS 配置。

### 1.3.2 音频设备配置

Audio APP 打开某一个 PCM 设备节点进行读写操作之前，需要先进行一些 mixer/control 的设置来打开 Audio Path，或者使能一些 Audio Hardware Interfaces。

下面以公版 MT8668 播放 Music 为例，通过 DL46 节点，打通 DL46→I2SOUT2→ES8311→Speaker 这条 Audio Path：

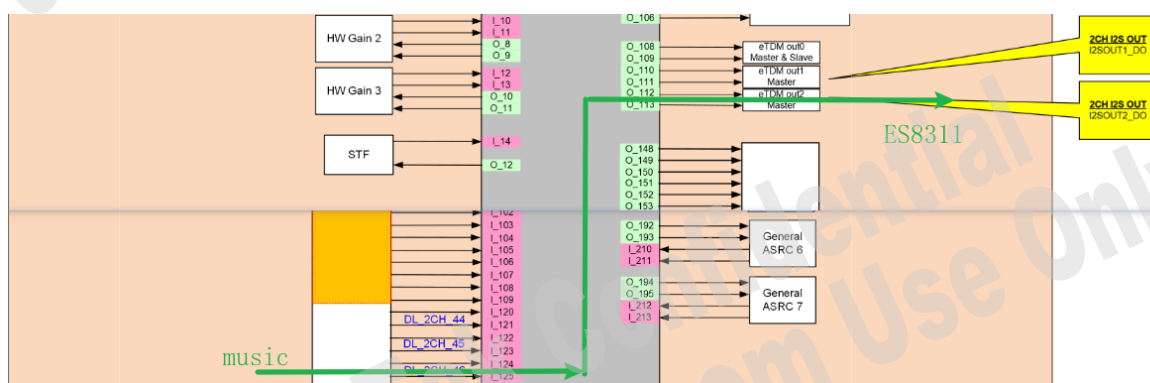


图 1-10. 播放音乐的音频路径

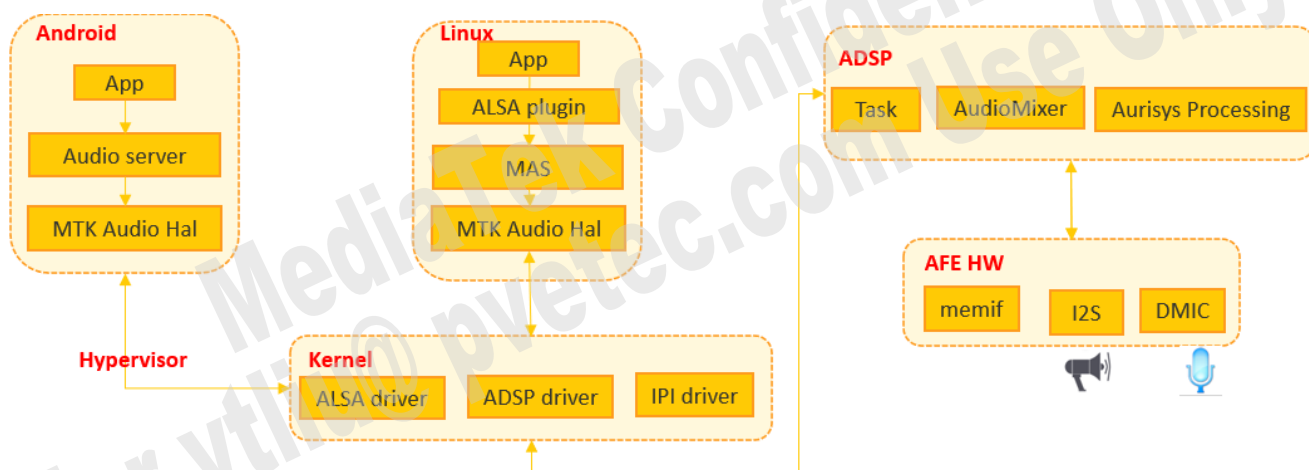
设定的配置 kcontrol 如下：

```
amixer cset name='I2S_OUT2_Mux' 'Dummy_Widget' //实际接了 es8311 codec 就不需要此 dummy widget 的 kcontrol
amixer cset name='I2SOUT2_CH1 DL46_CH1' 1
amixer cset name='I2SOUT2_CH2 DL46_CH2' 1
aplay -Dhw:0,11 -r48000 -c2 -fs16_LE /data/2ch.wav
```

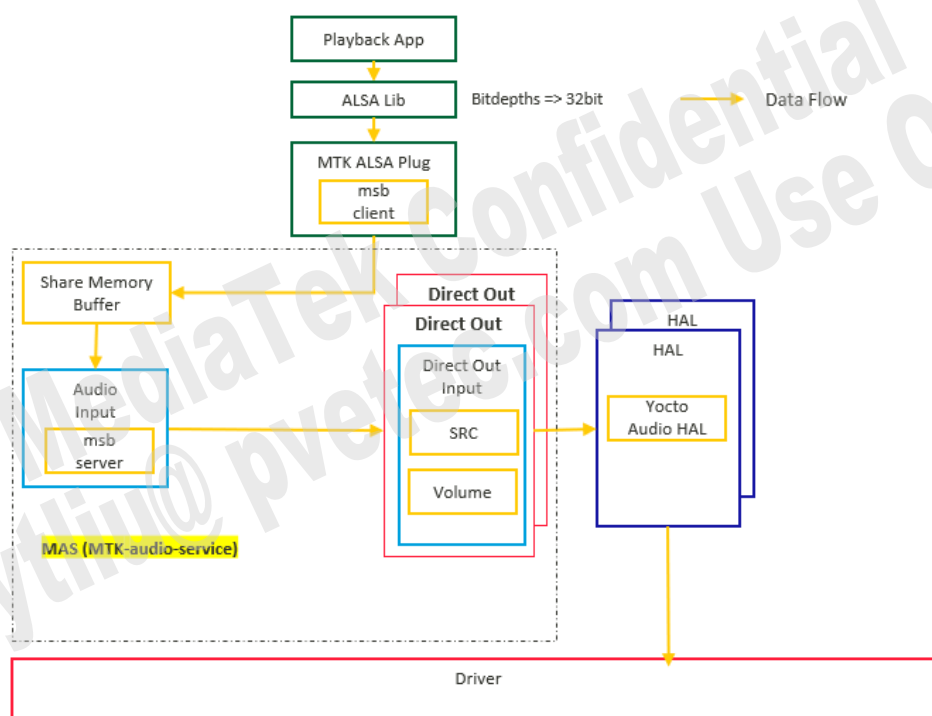
## 1.4 Yocto 播放 ADSP 音频

### 1.4.1 软件架构

前面介绍了基于 alsa lib 的软件架构，为了在 Yocto 侧也能播放 ADSP 节点，在用户层新增了 ALSA plugin，MAS，MTK Audio Hal 等模块，如下：



从 Ycoto 侧播放 ADSP，不需要走 Hypervisor 的虚拟化。



### 1.4.2 测试命令

为保证 APP 像正常调用 ALSA lib 一样方便，Yocto ADSP 播放提供 alsa 节点，有 main/sub0/direct 等节点，节点配置见 alsa config 文件，其中 main 和 sub0 是 mixer 节点，可以允许多路 app 混音播放，direct 是独立播放节点。

举例说明：

- Aplay 播放  
//混音播放

```
aplay -D main /data/music.wav
aplay -D sub0 /data/music.wav
```

#### //独立进程播放

```
aplay -D direct0 /data/music.wav
aplay -D direct1 /data/music.wav
```

- **Gstreamer 命令播放**

```
gst-launch-1.0 playbin audio-sink="alsasink device=main" uri=file:///data/test.wav
or
gst-play-1.0 /data/music.wav --audiosink="alsasinkdevice=main"
```

## 1.5 常见问题/故障排除

### 1.5.1 如何测试音频驱动?

当 Audio Driver 已经 ready 的时候, 可以使用 Yocto 自带的 aplay/arecord/amixer 来进行手动测试。其中, amixer 用来将 Audio Path 连通, aplay 用于播放声音, arecord 用来从 MIC 录音。以 i2s1 的 loopback 为例, 命令行如下:

```
adb -d push E:\MT8668\test\testwav\2ch.wav data/
adb shell
amixer cset name='I2SOUT1_CH1 DL1_CH1' 1
amixer cset name='I2SOUT1_CH2 DL1_CH2' 1
amixer cset name='I2S_OUT1_Mux' 'Dummy_Widget'
aplay -Dhw:0,1 -r48000 -c2 -fS16_LE /data/2ch.wav

amixer cset name='I2S_IN1_Mux' 'Dummy_Widget'
amixer cset name='UL1_CH1 I2SIN1_CH1' 1
amixer cset name='UL1_CH2 I2SIN1_CH2' 1
amixer cset name='I2SIN1_LPBK' 1
arecord -Dhw:0,15 -r48000 -c2 -fS16_LE /data/i2s1.wav
adb -d pull /data/i2s1.wav
```

Yocto 侧单独提供两个 AFE 节点, 一个用于播放, 另一个用于录音, 测试命令如下:

- **AFE 节点播放 DL46\_I2SOUT6(CH9/CH10)**

**#audio path:** DL46--->I2SOUT6(CH9/CH10)---->AK7709

```
amixer cset name='I2S_OUT6_Mux' 'Dummy_Widget'
amixer cset name='I2SOUT6_CH9 DL46_CH1' 1
amixer cset name='I2SOUT6_CH10 DL46_CH2' 1
aplay -Dhw:0,11 -r48000 -c2 -fS16_LE /data/2ch.wav
```

- **AFE 节点录音 I2SIN6(CH1/CH2)-->UL7**

**#audio path:** AK7709--->I2SIN6(CH1/CH2)---->UL7

```
amixer cset name='I2S_IN6_Mux' 'Dummy_Widget'
amixer cset name='UL7_CH1 I2SIN6_CH1' 1
```

```
amixer cset name='UL7_CH2 I2SIN6_CH2' 1
arecord -Dhw:0,21 -r48000 -c2 -fS16_LE /data/ul7.wav
```

如果没有接入 AK7709 codec，可以使用 sinetone 打入 I2SIN6\_CH1/CH2 来模拟录音。

```
echo write_reg,0x20,0x04F01F01 > /sys/kernel/debug/mtksocaudio
echo write_reg,0x24,0xa053 > /sys/kernel/debug/mtksocaudio
```

### 1.5.2 如何开启 log?

在开发或者解决 Bug 的时候，需要打开并查看 Audio Driver 的 log，来定位问题点。

可以用 `cat /sys/kernel/debug/dynamic_debug/control` 查看哪些文件可以打开 pr debug，再使用 `echo p` 的 command 打开，例如：

```
echo 'file mtk-afe-fe-dai.c +p' > /sys/kernel/debug/dynamic_debug/control
```

### 1.5.3 Audio Register 的查看

查看所有 Audio AFE register 的值，可以使用命令：

```
cat /sys/kernel/debug/mtksocaudio
```

例如像查看 ETDMOUT6 的 register，可以加上 `grep`：

```
cat /sys/kernel/debug/mtksocaudio | grep ETDM_OUT6
```

```
root@auto8668p164sos:/# cat /sys/kernel/debug/mtksocaudio | grep ETDM_OUT6
ETDM_OUT6_CON0 = 0x9ff800
ETDM_OUT6_CON1 = 0x200184c5
ETDM_OUT6_CON2 = 0x4f00406
ETDM_OUT6_CON3 = 0x76543210
ETDM_OUT6_CON4 = 0x400
ETDM_OUT6_CON5 = 0x0
ETDM_OUT6_CON6 = 0x55555555
ETDM_OUT6_CON7 = 0xaaaaaaaa
ETDM_OUT6_CON8 = 0xfedcba98
ETDM_OUT6_CON9 = 0x40004000
ETDM_OUT6_MON = 0x40004000
root@auto8668p164sos:/#
```

### 1.5.4 dts/kconfig/ko table/dws 文件的查看

Yocto 侧若有改动，需改动到正确的路径文件，总结如下：

|             |                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sos project | kernel dts                           | evb board dts路径: meta/meta-mediatek-mt8668-hyp/recipes-kernel/linux/files/sos_dts/auto8668p1_64_sos_evb.dts<br>demo board dts路径: meta/meta-mediatek-mt8668-hyp/recipes-kernel/linux/files/sos_dts/auto8668p1_64_sos.dts<br>dts也会吃到 mt6881.dts                                                                                                                                                                                                                                                                                                                       |
|             | kernel config<br>主要由多个config<br>合并而成 | config 主要由多个config文件合并,主要由下下面组成<br>1. deconfig src/kernel/linux/v6.12_mt8668/co_device_module/arch/arm64/configs/mgk_64_k612_defconfig<br>2. src/kernel/linux/v6.12_mt8668/co_device_module/kernel/configs/userdebug.config (仅仅build userdebug load 生效)<br>3. src/kernel/linux/v6.12_mt8668/co_device_module/kernel/configs/eng.config (仅仅build eng load 生效)<br>4. meta-mediatek-mt8668/recipes-kernel/linux/files/config/auto8668p1_64.config<br>5. <b>project config: meta-mediatek-mt8668-hyp/recipes-kernel/linux/files/sos_config/auto8668p1_64_sos.config</b> |
|             | hyp Kconfig 路径                       | sos: src/kernel/linux/v6.12_mt8668/co_device_module/drivers/hypervisor/sos/Kconfig                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|             | ko table                             | meta/meta-mediatek-mt8668-hyp/recipes-kernel/linux/ko_order_table/ko_order_table.csv                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|             | dws文件                                | src/devtools/dct/dws/mt6881/auto8668p1_64_sos.dws                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

## 附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。